

Data Grooming Requirements for Testing

Last updated by | Haroun Ahmady | Apr 15, 2026 at 10:10 AM PDT

Summary

Thank you for loading the **customer** and **account** data from the downstream systems. The Client tab is displaying customer information correctly.

However, we've identified that several critical tables are still empty, which is blocking key application features. Most urgently, the **Role-Based Access Control (RBAC)** tables need to be populated — without them, the Accounts tab is hidden and users cannot navigate to account details.

This document outlines what's needed, in priority order, with exact SQL you can run.

What's Currently Broken and Why

Issue	Root Cause
Accounts tab not visible	The <code>permission</code> and <code>role_permission</code> tables are empty. The UI checks for <code>accounts.view</code> permission — without it, the tab is hidden.
Clicking an account does nothing	Same cause — clicking tries to switch to Account Summary tab, which is also gated by <code>accounts.view</code> .
Household tab not visible	Missing <code>household.view</code> permission in RBAC tables.
No user roles assigned	The <code>employee_role</code> table is empty — no employees have any roles assigned.

Priority 0 — RBAC Tables (BLOCKER)

These 5 tables must be populated **before any role-gated feature works**. Load them in this exact order (foreign key dependencies).

Step 1: `privilege_level`

```
-- Defines the 5 access tiers (0-4)
INSERT INTO privilege_level (level, level_name, description) VALUES
(0, 'Read-Only', 'Minimal access. View-only on permitted resources.'),
(1, 'Staff', 'Basic customer and account operations. No admin access.'),
(2, 'Manager', 'Team management. Standard operational access.'),
(3, 'Senior/Branch', 'Can view employee customer records. Most operational access.'),
(4, 'System Admin', 'Everything. User/role management, SAML config, all data.');
```

Step 2: `role`

```
-- Create the 9 application roles
-- NOTE: created_by can be NULL for initial seed, or set to an existing employee_id
INSERT INTO role (role_name, privilege_level, description, is_system_role, is_active, created_at, updated_at)
('System Admin', 4, 'Full system access. Manages users, roles, SAML mappings.', 1, 1, GETDATE(), GE
('Branch Manager', 3, 'Branch-level authority. Can view employee customer data.', 1, 1, GETDATE(), G
('Assistant Manager', 2, 'Team management. Limited user management.', 1, 1, GETDATE(), G
('Loan Officer', 2, 'Lending operations. Account and customer access.', 1, 1, GETDATE(), G
('Business Banker', 2, 'Business relationship management.', 1, 1, GETDATE(), G
('Teller', 1, 'Day-to-day customer and account operations.', 1, 1, GETDATE(), G
('Customer Service Rep', 1, 'Basic customer lookup and service.', 1, 1, GETDATE(), G
('Risk Analyst', 1, 'Risk monitoring and review.', 1, 1, GETDATE(), G
('Compliance Officer', 1, 'Compliance monitoring.', 1, 1, GETDATE(), G
```

Step 3: permission

```
-- Create all 11 permission codes
-- min_privilege_level: if set, any user at or above this level automatically gets this permission
-- is_attribute_based: 1 = has ABAC conditions (checked at runtime)
INSERT INTO permission (permission_code, resource, action, description, min_privilege_level, is_attribute_base
('accounts.view', 'accounts', 'view', 'View customer accounts and details'
('account.view.balances', 'account', 'view.balances', 'View balance and interest rate colu
('transaction.view', 'transaction', 'view', 'View transaction history',
('customer.view.relationship_summary', 'customer', 'view.relationship_summary', 'View relationship summ
('customer.view.recent_activity', 'customer', 'view.recent_activity', 'View recent activity c
('customer.view.deposits', 'customer', 'view.deposits', 'View deposits section',
('household.view', 'household', 'view', 'View household tab and relationship
('users.view', 'users', 'view', 'View user list, details, and roles
('users.assign_roles', 'users', 'assign_roles', 'Assign or remove roles from users',
('user_management.view', 'user_management', 'view', 'View SAML role mappings',
('user_management.assign_roles', 'user_management', 'assign_roles', 'Create, update, delete SAML role ma
```

Important notes on min_privilege_level:

- A permission with min_privilege_level = 2 is **automatically granted** to any user with privilege level 2, 3, or 4 — even without an explicit role_permission entry.
- Level 1 (Staff) roles like Teller only get permissions explicitly assigned via role_permission.
- Set min_privilege_level = NULL if you want the permission to be exclusively role-assigned.

Step 4: role_permission

This maps which roles get which permissions explicitly. Use this to grant Level 1 roles (Teller, CSR, etc.) access to features they need.

```

-- First, get the role and permission IDs
-- Adjust these IDs based on your actual auto-generated IDs after Steps 2 and 3

-- Grant Level 1 roles explicit permissions they need
-- (Level 2+ roles already inherit via min_privilege_level on the permission)

-- Example: Grant Teller and Customer Service Rep the accounts.view permission
INSERT INTO role_permission (role_id, permission_id, granted_by, granted_at)
SELECT r.role_id, p.permission_id, NULL, GETDATE()
FROM role r
CROSS JOIN permission p
WHERE r.role_name IN ('Teller', 'Customer Service Rep')
      AND p.permission_code IN (
        'accounts.view',
        'account.view.balances',
        'transaction.view',
        'customer.view.relationship_summary',
        'customer.view.recent_activity',
        'customer.view.deposits',
        'household.view'
      );

-- Grant Risk Analyst and Compliance Officer read-only account access
INSERT INTO role_permission (role_id, permission_id, granted_by, granted_at)
SELECT r.role_id, p.permission_id, NULL, GETDATE()
FROM role r
CROSS JOIN permission p
WHERE r.role_name IN ('Risk Analyst', 'Compliance Officer')
      AND p.permission_code IN (
        'accounts.view',
        'account.view.balances',
        'customer.view.relationship_summary',
        'customer.view.recent_activity',
        'customer.view.deposits',
        'household.view'
      );

```

Step 5: employee_role

Assign roles to the test employees so they can log in and see features.

```

-- Assign roles to test employees
-- Replace @employeeId with actual employee IDs from your loaded data
-- Replace @roleId with actual role IDs from Step 2

-- Example: Assign "System Admin" role to a test employee (full access for testing)
DECLARE @sysAdminRoleId INT = (SELECT role_id FROM role WHERE role_name = 'System Admin');
DECLARE @branchMgrRoleId INT = (SELECT role_id FROM role WHERE role_name = 'Branch Manager');
DECLARE @tellerRoleId INT = (SELECT role_id FROM role WHERE role_name = 'Teller');

-- Pick a few employees to assign roles to for testing
-- System Admin – sees everything
INSERT INTO employee_role (employee_id, role_id, is_primary, assigned_date, effective_date, expiration_date, i
SELECT TOP 1 employee_id, @sysAdminRoleId, 1, GETDATE(), GETDATE(), NULL, 1
FROM employee WHERE is_active = 1;

-- Branch Manager – sees employee customer data
INSERT INTO employee_role (employee_id, role_id, is_primary, assigned_date, effective_date, expiration_date, i
SELECT TOP 1 employee_id, @branchMgrRoleId, 1, GETDATE(), GETDATE(), NULL, 1
FROM employee WHERE is_active = 1 AND employee_id NOT IN (SELECT employee_id FROM employee_role);

-- Teller – restricted from employee customer records
INSERT INTO employee_role (employee_id, role_id, is_primary, assigned_date, effective_date, expiration_date, i
SELECT TOP 1 employee_id, @tellerRoleId, 1, GETDATE(), GETDATE(), NULL, 1
FROM employee WHERE is_active = 1 AND employee_id NOT IN (SELECT employee_id FROM employee_role);

-- Bulk-assign remaining employees as Tellers (or adjust as needed)
INSERT INTO employee_role (employee_id, role_id, is_primary, assigned_date, effective_date, expiration_date, i
SELECT employee_id, @tellerRoleId, 1, GETDATE(), GETDATE(), NULL, 1
FROM employee
WHERE is_active = 1
AND employee_id NOT IN (SELECT employee_id FROM employee_role);

```

Priority 1 — Core Relationship Tables

These tables enable the Household tab, account ownership display, and officer assignments.

Table	Purpose	Key Columns	Foreign Keys
region	Geographic regions for branches	region_id , region_name , region_code	—
branch	Bank branches	branch_id , branch_code , branch_name , region_id	region_id → region
account_ownership	Links customers to accounts	account_id , customer_id , ownership_type , is_primary_owner	account_id → account , customer_id → customer
household	Customer groupings	household_id , household_name , relationship_manager_id	relationship_manager_id → employee
household_membership	Members of households	household_id , customer_id , relationship_role , is_primary_member	household_id → household , customer_id → customer
customer_officer_assignment	Assigns officers to customers	customer_id , officer_code , relationship_type	customer_id → customer

Critical: The `account_ownership` table is essential. Without it, clicking a customer won't show their accounts. Each customer-account relationship needs a row here with `is_primary_owner = 1` for at least one owner.

Priority 2 — Feature Enhancement Tables

These enable transactions, debit cards, and industry classification features.

Table	Purpose	Key Columns	Depends On
transaction_category	Categories like ATM, wire, ACH	category_id , name , group_code	—
financial_transaction	Transaction history	account_id , amount , transaction_type , transaction_date	account , transaction_category
debit_card_limit_profile	Card spending limits	profile_id , profile_name , daily_purchase_limit	—
debit_card	Debit cards on checking accounts	account_id , customer_id , card_status , last_four_digits	account (checking only), customer , debit_card_limit_profile
sic_code	Industry classification codes	sic_code , description	—
customer_sic_code	Customer industry mapping	customer_id , sic_code	customer , sic_code
account_sic_code	Account industry mapping	account_id , sic_code	account , sic_code
contact_history	Customer interaction log	customer_id , contact_type , occurred_at	customer , employee
entity_address	Links addresses to customers	entity_type , entity_id , address_id	address
entity_contact	Links contact info to customers	entity_type , entity_id , contact_id	contact_info
address	Physical addresses	address_id , address_line1 , city , state	—

Table	Purpose	Key Columns	Depends On
contact_info	Phone, email, fax	contact_id , contact_type , contact_value	—

Priority 3 — Notes & Audit Tables

Table	Purpose	Depends On
note_category	Note classification	—
note	Customer/account notes	customer OR account , note_category
note_version	Versioned note content	note , employee
online_banking_user	Online banking credentials	customer
online_banking_login_event	Login audit trail	online_banking_user

Test Scenarios Enabled by Data

Scenario	Required Data
Basic login and view customer	employee + employee_role + RBAC tables
See Accounts tab	Above + accounts.view permission granted
View account details	Above + account_ownership linking customer to account
Household tab	household + household_membership + household.view permission
Employee customer protection	A customer with is_employee = 1 + a Level 1 user (should be blocked) + a Level 3 user (should see data)
Transaction history	financial_transaction ROWS + transaction.view permission
Debit card display	debit_card + debit_card_limit_profile (checking accounts only)
Admin user management	users.view permission (Level 3+)

Validation Queries

Run these after loading to verify everything is wired up correctly:

```
-- 1. Verify RBAC chain is complete for a specific employee
DECLARE @testEmployeeId BIGINT = (SELECT TOP 1 employee_id FROM employee_role WHERE is_active = 1);

SELECT
    e.first_name + ' ' + e.last_name AS employee_name,
    r.role_name,
    r.privilege_level,
    STRING_AGG(p.permission_code, ', ') AS explicit_permissions
FROM employee e
JOIN employee_role er ON e.employee_id = er.employee_id
JOIN role r ON er.role_id = r.role_id
LEFT JOIN role_permission rp ON r.role_id = rp.role_id
LEFT JOIN permission p ON rp.permission_id = p.permission_id
WHERE e.employee_id = @testEmployeeId
    AND er.is_active = 1
    AND r.is_active = 1
GROUP BY e.first_name, e.last_name, r.role_name, r.privilege_level;

-- 2. Check which permissions a user gets via privilege level (inherited)
SELECT permission_code, min_privilege_level
FROM permission
WHERE is_active = 1
    AND min_privilege_level IS NOT NULL
ORDER BY min_privilege_level;

-- 3. Verify account_ownership exists (required for account display)
SELECT
    COUNT(*) AS total_ownerships,
    COUNT(DISTINCT account_id) AS accounts_with_owners,
    COUNT(DISTINCT customer_id) AS customers_with_accounts
FROM account_ownership
WHERE is_active = 1;

-- 4. Check if any employee has no role assigned (they won't be able to use the app)
SELECT e.employee_id, e.first_name, e.last_name
FROM employee e
LEFT JOIN employee_role er ON e.employee_id = er.employee_id AND er.is_active = 1
WHERE e.is_active = 1
    AND er.employee_id IS NULL;

-- 5. Verify the critical accounts.view permission exists and is active
SELECT * FROM permission WHERE permission_code = 'accounts.view';
```

Quick Reference: Complete Table Load Order

Load in this order to respect foreign key constraints:

1. privilege_level
2. region
3. branch (needs region)
4. address
5. contact_info
6. employee (needs branch)
7. role (needs privilege_level)
8. permission (needs privilege_level for min_privilege_level)

9. role_permission (needs role + permission)
10. employee_role (needs employee + role)
11. employee_branch (needs employee + branch)
12. customer (needs branch)
13. entity_address (needs address)
14. entity_contact (needs contact_info)
15. household (needs employee for relationship_manager_id)
16. household_membership (needs household + customer)
17. account (needs branch)
18. account_ownership (needs account + customer)
19. sic_code
20. customer_sic_code (needs customer + sic_code)
21. account_sic_code (needs account + sic_code)
22. customer_officer_assignment (needs customer)
23. debit_card_limit_profile
24. debit_card (needs account + customer + debit_card_limit_profile)
25. transaction_category
26. financial_transaction (needs account + transaction_category)
27. contact_history (needs customer + employee)
28. note_category
29. note (needs customer or account)
30. note_version (needs note + employee)
31. online_banking_user (needs customer)
32. online_banking_login_event (needs online_banking_user)

Please reach out if you have questions about column types or constraints. The full schema is in `shared/schema.ts` and the roles documentation is in `docs/roles-and-permissions.md`.